AUTOMATION · DOCUMENTATION

Portfolio Assistant

AI assistant · inquiry automation

A chatbot that answers recruiter and client questions about my projects, ownership, and availability, grounded in the same data that powers this site, and hands off to the contact form when it can't answer.

ROLE	BUILT	STATUS
Solo build	**Oct 2026**	**Built into this site**

CONTENTS

1. Overview
2. My contribution
3. Workflow, step by step
4. Safeguards and exception handling
5. Technology stack
6. Outcomes
7. Known limits

Prince John Gandollas · Full-Stack Engineer
pgandollas@gmail.com · LinkedIn · GitHub · pjdevph.vercel.app

Generated from the portfolio's content on 2026-10-12. Live version: pjdevph.vercel.app/automations/portfolio-assistant

1. Overview

A chatbot that answers recruiter and client questions about my projects, ownership, and availability, grounded in the same data that powers this site, and hands off to the contact form when it can't answer.

AUTOMATION	Portfolio Assistant
TYPE	AI automation
STATUS	implemented (Built into this site)
ROLE	Solo build
BUILT	Oct 2026
PROBLEM	Visitors skim. The questions that matter, such as what I personally owned, whether something is live, and whether I'm available, are spread across several pages or only answered by email.
MANUAL PROCESS REPLACED	A visitor reads multiple case studies or emails me, and I answer repeat questions by hand.
TRIGGER	A visitor opens the chat and asks a question.
SOLUTION	A streaming assistant that answers from a verified knowledge base, separates platform scale from my own contribution, and points to the contact form instead of guessing.

2. My contribution

Designed and built end to end: knowledge base, guardrails, API route, streaming UI, and this case study.

3. Workflow, step by step

#	Step	What happens
1	Question	Visitor asks in the chat or taps a suggested prompt. Common questions (contact, skills, projects, availability, technologies) are answered instantly from a built-in FAQ, for free.
2	Guardrails	Validates length and history, checks the request origin, applies per-visitor and daily rate limits, and can require a Cloudflare Turnstile check.
3	Grounding	The system prompt is built from the same project, experience and automation data the site renders, so there is one source of truth.
4	Claude	Open-ended questions go to Claude Haiku 5.5 under strict rules at low effort; the knowledge base prefix is cached. A monthly budget is reserved before each call.

#	Step	What happens
5	Streamed answer	Text streams to the page as it is generated.
6	Hand-off	Unknown or hiring questions are routed to the contact form or email.

4. Safeguards and exception handling

Grounded answers only

Rules require answering only from the knowledge base, never inventing employers, dates, metrics, or links, and saying so when it doesn't know.

Scope vs. ownership

Platform counts (for example 114 Edge Functions) are described as platform scale; my own contributions come from the listed areas of ownership.

Untrusted input

Visitor text can't change the rules, roles are validated server-side, and only user and assistant turns are accepted from the browser.

Abuse and cost limits

Messages are capped at 600 characters and 12 turns, with per-visitor and daily limits, a hard monthly spend budget enforced in code, an optional Redis-backed store so limits hold across servers, an optional Turnstile check, and a same-site origin check.

Answer-first FAQ and free fallback

Common questions are answered from a built-in rule-based FAQ built from the same site data before any AI call. The same FAQ takes over when the monthly budget is spent, the API key is missing, or the provider fails, and the chat says so. It costs nothing and cannot invent facts, but it only covers common questions.

Hard monthly budget

Before each model call a worst-case cost is reserved against a monthly budget (default $2). If it would be exceeded, the visitor gets the free FAQ instead. Actual cost replaces the reservation afterwards, and failed calls release it.

Graceful failure

Refusals, rate limits, and mid-answer failures each show a clear message with a link to the contact form.

Privacy and logging

This site doesn't store conversations. Logs hold token counts and estimated cost only, never message text or IP addresses. Messages are processed by the model provider's API to produce a reply.

5. Technology stack

- Next.js route handler
- Anthropic TypeScript SDK
- Claude (claude-haiku-5-5, configurable)
- Streaming NDJSON
- TypeScript
- Tailwind CSS

6. Outcomes

- Implemented as a floating assistant on every page and an embedded demo on this page.
- Streaming, validation, rate limiting, refusal handling, and error paths were verified against a stand-in model server.
- The free FAQ answers 98.7% of 776 generated recruiter phrasings correctly in an offline test, with no AI call. Answer quality, grounding and real cost of the model path have not been measured against the live API yet. A repeatable evaluation suite is included for that.
- No usage, conversion, or time-saved figures are claimed. The site doesn't collect them.

7. Known limits

- Answers can be wrong or incomplete; it only knows what is on this portfolio.
- Prompt-injection defenses reduce risk but can't remove it.
- Without the optional Redis store, rate limits and the monthly budget counter are per server instance, so they slow abuse rather than guarantee a hard cap. A spend limit on the provider key is the final backstop.
- The built-in FAQ only covers common questions and tested phrasings; unusual wording falls through to the model, or to a polite "I don't have that" when the model is off.
- No memory between visits and no human-handoff inbox beyond the contact form.

> This document describes architecture and behaviour only. It contains no credentials, internal endpoints or customer data.